1-Number of Zeros in a Given Array

Started on Thursday, 27 August 2026, 10:38 AM

State Finished

Completed on Thursday, 27 August 2026, 10:39 AM

Time taken 53 secs

Marks 1.00/1.00

Grade **10.00** out of 10.00 (**100**%)

Question 1 Correct Mark 1.00 out of 1.00 Flag question

Problem Statement

Given an array of 1s and 0s this has all 1s first followed by all 0s. Aim is to find the number of 0s. Write a program using Divide and Conquer to Count the number of zeroes

Input Format

First Line Contains Integer m – Size of array

Next m lines Contains m numbers – Elements of an array

Output Format

First Line Contains Integer – Number of zeroes present in the given array.

Answer: (penalty regime: 0 %)

```
#include <stdio.h>
#include <stdlib.h>

int countZeros(int* arr, int low, int high, int m) {
    int firstZeroIndex = m;

```

```c
    while (low <= high) {
        int mid = low + (high - low) / 2;

        if (arr[mid] == 0) {
            firstZeroIndex = mid;
            high = mid - 1;
        } else {
            low = mid + 1;
        }
    }

    return m - firstZeroIndex;
}

int main() {
    int m;
    if (scanf("%d", &m) != 1 || m <= 0) return 0;

    int* arr = (int*)malloc(m * sizeof(int));
    for (int i = 0; i < m; i++) {
        scanf("%d", &arr[i]);
    }

    int zeros = countZeros(arr, 0, m - 1, m);

    printf("%d\n", zeros);

    free(arr);
    return 0;
}
```

	Input	Expected	Got	
✔	5 1 1 1	2	2	✔

✔	8 0 0 0 0 0 0 0 0	8	8	✔
✔	17 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0	2	2	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

2-Majority Element

Started on	Thursday, 27 August 2026, 10:39 AM
State	Finished
Completed on	Thursday, 27 August 2026, 10:40 AM
Time taken	34 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 Correct Mark 1.00 out of 1.00 Flag question

Given an array `nums` of size `n`, return *the majority element*.

The majority element is the element that appears more than `⌊n / 2⌋` times. You may assume that the majority element always exists in the array.

Example 1:

```
Input: nums = [3,2,3]
Output: 3
```

Example 2:

```
Input: nums = [2,2,1,1,1,2,2]
Output: 2
```

Constraints:

For example:

Input	Result
3 3 2 3	3
7 2 2 1 1 1 2 2	2

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdlib.h>

int findMajorityElement(int* nums, int numsSize) {
    int candidate = 0;
    int count = 0;

    for (int i = 0; i < numsSize; i++) {
        if (count == 0) {
            candidate = nums[i];
            count = 1;
        } else if (nums[i] == candidate) {
            count++;
        } else {
            count--;
        }
    }

    return candidate;
}

int main() {
    int n;
    if (scanf("%d", &n) != 1 || n <= 0) return 0;

    int* nums = (int*)malloc(n * sizeof(int));
    if (nums == NULL) return 1;

    for (int i = 0; i < n; i++) {
        if (scanf("%d", &nums[i]) != 1) {
```

	Input	Expected	Got	
✔	3 3 2 3	3	3	✔

Passed all tests! ✔

2-Majority Element

Started on	Thursday, 27 August 2026, 10:39 AM
State	Finished
Completed on	Thursday, 27 August 2026, 10:40 AM
Time taken	34 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct | Mark 1.00 out of 1.00 | Flag question

Input	Result
3 3 2 3	3
7 2 2 1 1 1 2 2	2

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdlib.h>

int findMajorityElement(int* nums, int numsSize) {
    int candidate = 0;
    int count = 0;

    for (int i = 0; i < numsSize; i++) {
        if (count == 0) {
            candidate = nums[i];
            count = 1;
        } else if (nums[i] == candidate) {
            count++;
        } else {
            count--;
        }
    }

    return candidate;
}

int main() {
    int n;
    if (scanf("%d", &n) != 1 || n <= 0) return 0;

    int* nums = (int*)malloc(n * sizeof(int));
    if (nums == NULL) return 1;

    for (int i = 0; i < n; i++) {
        if (scanf("%d", &nums[i]) != 1) {
            free(nums);
            return 0;
```

```
    for (int i = 0; i < n; i++) {
        if (scanf("%d", &nums[i]) != 1) {
            free(nums);
            return 0;
        }
    }

    int result = findMajorityElement(nums, n);
    printf("%d\n", result);

    free(nums);
    return 0;
}
```

	Input	Expected	Got	
✔	3 3 2 3	3	3	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

3-Finding Floor Value

Started on Thursday, 17 September 2026, 11:29 AM

State Finished

Completed on Thursday, 17 September 2026, 11:33 AM

Time taken 3 mins 38 secs

Marks 1.00/1.00

Grade **10.00** out of 10.00 (**100**%)

Question 1 Correct Mark 1.00 out of 1.00 Flag question

Problem Statement:

Given a sorted array and a value x, the floor of x is the largest element in array smaller than or equal to x. Write divide and conquer algorithm to find floor of x.

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Value for x

Output Format

First Line Contains Integer – Floor value for x

Answer: (penalty regime: 0 %)

```
#include <stdio.h>
#include <stdlib.h>

int findFloor(int arr[], int n, int x) {
```

```c
#include <stdio.h>
#include <stdlib.h>

int findFloor(int arr[], int n, int x) {
    int low = 0;
    int high = n - 1;
    int floorVal = -1;

    while (low <= high) {
        int mid = low + (high - low) / 2;

        if (arr[mid] == x) {
            return arr[mid];
        } else if (arr[mid] < x) {
            floorVal = arr[mid];
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }

    return floorVal;
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    int *arr = (int *)malloc(n * sizeof(int));
    for (int i = 0; i < n; i++) {
        if (scanf("%d", &arr[i]) != 1) {
            free(arr);
            return 0;
        }
    }

    int x;
    if (scanf("%d", &x) != 1) {
        free(arr);
        return 0;
    }

    printf("%d\n", findFloor(arr, n, x));

    free(arr);
```

	Input	Expected	Got	
✔	6 1 2 8 10 12 19 5	2	2	✔
✔	5 10 22 85 108 129 100	85	85	✔
✔	7 3 5 7 9 11 13 15 10	9	9	✔

Passed all tests! ✔

4-Two Elements sum to x

Started on Thursday, 27 August 2026, 10:41 AM

State Finished

Completed on Thursday, 27 August 2026, 10:41 AM

Time taken 42 secs

Marks 1.00/1.00

Grade **10.00** out of 10.00 (**100**%)

Question 1 Correct Mark 1.00 out of 1.00 Flag question

Problem Statement:

Given a sorted array of integers say arr[] and a number x. Write a recursive program using divide and conquer strategy to check if there exist two elements in the array whose sum such two elements then return the numbers, otherwise print as "No".

Note: Write a Divide and Conquer Solution

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Sum Value

Output Format

First Line Contains Integer – Element1

Second Line Contains Integer – Element2 (Element 1 and Elements 2 together sums to value "x")

Answer: (penalty regime: 0 %)

```
#include <stdio.h>
#include <stdlib.h>

int findPair(int* arr, int low, int high, int x, int* num1, int* num2) {
```

```c

int findPair(int* arr, int low, int high, int x, int* num1, int* num2) {
    if (low >= high) {
        return 0;
    }

    int sum = arr[low] + arr[high];
    if (sum == x) {
        *num1 = arr[low];
        *num2 = arr[high];
        return 1;
    } else if (sum < x) {
        return findPair(arr, low + 1, high, x, num1, num2);
    } else {
        return findPair(arr, low, high - 1, x, num1, num2);
    }
}

int main() {
    int n;
    if (scanf("%d", &n) != 1 || n <= 0) return 0;

    int* arr = (int*)malloc(n * sizeof(int));
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    int x;
    if (scanf("%d", &x) != 1) {
        free(arr);
        return 0;
    }

    int num1, num2;
    int found = findPair(arr, 0, n - 1, x, &num1, &num2);

    if (found) {
        printf("%d\n%d\n", num1, num2);
    } else {
        printf("No\n");
    }

    free(arr);
    return 0;
}
```

	Input	Expected	Got	
✔	4 2 4 8 10 14	4 10	4 10	✔
✔	5 2 4 6 8 10 100	No	No	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

5-Implementation of Quick Sort

Started on Thursday, 27 August 2026, 10:42 AM

State Finished

Completed on Thursday, 17 September 2026, 10:26 AM

Time taken 20 days 23 hours

Marks 1.00/1.00

Grade **10.00** out of 10.00 (**100%**)

Question 1 | Correct Mark 1.00 out of 1.00 | Flag question

Write a Program to Implement the Quick Sort Algorithm

Input Format:
The first line contains the no of elements in the list-n
The next n lines contain the elements.

Output:
Sorted list of elements

For example:

Input	Result
5 67 34 12 98 78	12 34 67 78 98

```c
#include <stdio.h>
#include <stdlib.h>

void swap(int* a, int* b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return i + 1;
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

int main() {
    int n;
    if (scanf("%d", &n) != 1 || n <= 0) {
        return 0;
    }

    int* arr = (int*)malloc(n * sizeof(int));
    if (arr == NULL) {
        return 1;
    }

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    quickSort(arr, 0, n - 1);
```

```c
    quickSort(arr, 0, n - 1);

    for (int i = 0; i < n; i++) {
        printf("%d%s", arr[i], (i == n - 1) ? "" : " ");
    }
    printf("\n");

    free(arr);
    return 0;
}
```

	Input	Expected	Got	
✔	5 67 34 12 98 78	12 34 67 78 98	12 34 67 78 98	✔
✔	10 1 56 78 90 32 56 11 10 90 114	1 10 11 32 56 56 78 90 90 114	1 10 11 32 56 56 78 90 90 114	✔
✔	12 9 8 7 6 5 4 3 2 1 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.